CSE 318 Offline 1 — N-Puzzle (A*)

Viva Preparation: 20 Questions with Short Answers

1. Why is A* used instead of plain BFS/DFS for the n-puzzle?

BFS explores all nodes level by level regardless of quality, and the branching factor makes it blow up in memory/time. A* uses g(n)+h(n) to prioritize promising nodes, so it explores far fewer states while still guaranteeing optimality if h(n) is admissible. DFS doesn't guarantee shortest path at all.

2. What does "admissible heuristic" mean, and why does A* need it?

A heuristic is admissible if it never overestimates the true cost to reach the goal. If h(n) overestimates, A* might prune a node that's actually on the optimal path, breaking optimality guarantees. Admissibility ensures f(n) = g(n)+h(n) never exceeds the true cost, so the first goal popped is optimal.

3. Why is Manhattan distance a better heuristic than Hamming distance?

Hamming only counts misplaced tiles (0 or 1 per tile), ignoring how far each tile actually is. Manhattan sums actual row+column distances, giving a tighter (larger but still admissible) lower bound. A tighter bound means A* explores fewer nodes to reach the same optimal answer.

4. Is Euclidean distance admissible for this puzzle? Why or why not?

Yes — straight-line distance is always less than or equal to the actual grid-move distance since moves are constrained to horizontal/vertical steps. But it's a looser bound than Manhattan distance (since diagonal shortcuts don't exist in the puzzle), so it explores more nodes than Manhattan despite both being admissible.

5. Explain the intuition behind Linear Conflict as an enhancement to Manhattan distance.

Manhattan distance assumes tiles can pass through each other. If two tiles are in their correct row/column but swapped in order, one must temporarily move out of that line and back in — costing at least 2 extra moves. Linear conflict detects such pairs and adds 2 moves per conflict, tightening the estimate while staying admissible.

6. Why does the linear conflict formula add exactly 2 times conflicts, not 1x or 3x?

To let one tile pass the other, at least one tile must step out of the line (1 extra move) and then return into the line (1 more move) — a net of 2 extra moves

beyond straight Manhattan movement. This is the minimum extra cost, so it stays a valid lower bound (admissible).

7. What happens to A*'s performance if you use a non-admissible heuristic?

It may find a solution faster (fewer nodes expanded) but the solution is no longer guaranteed optimal — it could overestimate cost for some node and let a worse path get expanded to goal first. This is essentially what Weighted A* deliberately exploits.

8. In Weighted A*, what's the effect of increasing W?

Increasing W puts more trust in the heuristic and less in the guaranteed cost-so-far, making search more greedy/focused toward the goal. This drastically reduces nodes explored but sacrifices optimality — solution cost can exceed the true minimum, and the gap grows as W increases.

9. If W = 1 in Weighted A*, what algorithm do you get back?

Plain A*, since f(n) = g(n) + 1 times h(n) is exactly the standard priority function. W=1 is the boundary case that still preserves optimality (given an admissible h).

10. What would happen if W tends to infinity?

The search becomes essentially greedy best-first search, ignoring g(n) almost entirely and always expanding the node with lowest h(n). It finds a goal fast but the path can be far from optimal, since it never accounts for cost already paid.

11. Why do we need a closed list in A*?

To avoid re-expanding states already processed, which would cause infinite loops or redundant work since the same board configuration can be reached via multiple move sequences. Once a node is expanded with its lowest-cost path, revisiting it can't improve on that (assuming a consistent heuristic).

12. What is an inversion, and why does its parity determine solvability?

An inversion is a pair of tiles out of their relative goal order when the board is read row-major. Each legal slide move changes the inversion count's parity in a predictable way tied to blank movement, so only configurations reachable from the goal via valid moves share the same parity class as the goal — this partitions all boards into two disjoint equivalence classes (solvable / unsolvable).

13. Why does the solvability rule differ for even vs odd grid size k?

For odd k, the blank always returns to the same row after an even number of moves regardless of direction, so inversion parity alone determines solvability. For even k, moving the blank vertically changes which row it's in, which itself flips required inversion parity, so the blank's row (from bottom) must be factored in along with inversion count.

14. If you swap two tiles (not involving the blank) in a solved puzzle, is the new configuration solvable?

No — swapping any two non-blank tiles changes the inversion count by exactly 1 (odd), flipping its equivalence class. Since the goal state has 0 inversions (even, solvable class), a single swap always produces an unsolvable configuration for odd k.

15. Why must the heuristic logic be separated from the core A* search logic in your implementation?

This is a software design requirement for modularity — the priority function calls h(n) but shouldn't care which formula computes it. Using something like a strategy pattern (passing the heuristic function as a parameter) lets you switch heuristics or add custom ones without touching the search loop, priority queue, or node expansion logic.

16. Why is a priority queue used instead of a simple queue or stack for the open list?

The core requirement of A* is always expanding the node with the lowest f(n) next. A priority queue gives O(log n) insertion and O(log n) extraction of the minimum, which is essential for efficiency — a plain queue/stack would require full scans (O(n)) to find the minimum each time.

17. What is the role of the "previous node reference" stored in each search node?

It lets you reconstruct the full solution path by backtracking from the goal node to the initial node once the goal is dequeued, without storing the entire path at every node (which would waste memory). It's essentially a parent pointer for path reconstruction.

18. If two different heuristics are both admissible, how do you compare their quality?

The heuristic that gives a higher (closer to true cost) estimate for every state without exceeding it "dominates" the other — it's more informed and causes A* to

expand fewer nodes. So among admissible heuristics, tighter is always better for efficiency, though all yield the same optimal solution cost.

19. Why is the number of nodes explored expected to increase as puzzle size k grows, even with a good heuristic?

The state space grows factorially with k squared (roughly (k squared)!/2 reachable states), and even a tight heuristic can't perfectly predict true cost, so uncertainty compounds over a larger search space, forcing more expansions to find the optimal path.

20. Why do you need to verify solvability before running A*, instead of just letting A* run and detect it can't find a solution?

An unsolvable board has no path to goal, so A* would exhaust the entire reachable equivalence class (potentially exploring millions of nodes or running indefinitely on larger boards) before concluding no solution exists. Checking inversion parity beforehand is O(n squared) and instantly rules this out, avoiding wasted computation.